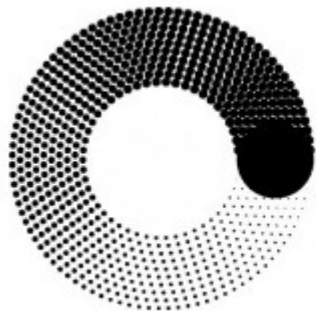




**МОСКОВСКИЙ
ПОЛИТЕХ**

**АВТОМАТИЗАЦИЯ ПРОЦЕССОВ ЖИЗНЕННОГО
ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ**

**Лабораторная работа № 7.
Автоматизированное тестирование**

**Москва
2024**

Теоретические основы

Автоматизированное тестирование лежит в основе любого конвейера CI/CD. Это связано с тем, что вся концепция CI/CD вращается вокруг принципа «быстро строить, быстро тестировать, быстро ошибаться».

Тесты должны выполняться как можно быстрее, чтобы результаты доходили до разработчика как можно раньше. Это позволяет своевременно обнаруживать ошибки. В результате продукт, выходящий на стадию релиза, будет более качественным и лишенным очевидных ошибок. Все это будет возможно только с автоматизацией тестирования ПО.

Автоматизация тестирования дает следующие преимущества:

1. Сокращает ручные усилия, когда один и тот же тест необходимо запускать повторно. Это освобождает больше времени для выполнения тщательных ручных тестов.
2. Дает немедленную обратную связь.
3. Тесты могут быть более точными и охватывать больше тестовых случаев.
4. Помогает генерировать и сравнивать результаты нескольких тестов, обеспечивая согласованность продукта.
5. Становится неотъемлемой частью конвейера CI/CD, соблюдая принцип «быстро строить, быстро ошибаться».

Очень важно иметь несколько наборов автоматизированных тестов, созданных для разных целей. Запуск одного набора тестов, охватывающего все сценарии, только замедлит процесс.

Ниже приведены некоторые типы тестирования и их место в конвейерах CI/CD:

Юнит тесты

Многие команды используют подход Test Driven Development (TDD). Юнит тесты пишутся разработчиками и запускаются сразу после этапа сборки. **Модули тестируются отдельно друг от друга.**

Интеграционные тесты

После сборки и развертывания каждого коммита в среде разработки эти тесты запускаются, чтобы проверить, хорошо ли взаимодействуют новые добавленные модули/изменения. **Тестируется совместная работа модулей**

Регрессионные тесты

Регрессионные тесты запускаются один раз в некоторое время (например, после окончания рабочего дня), чтобы гарантировать, что новые добавленные изменения не повлияют на существующий код. Это помогает убедиться, что дневная работа прошла хорошо, и дает обратную связь, если необходимы какие-либо изменения. **Все, что было написано ранее, тестируется, даже если оно было протестировано недавно.**

Нагрузочные тесты

Перед выпуском кода в производственную среду выполняется набор тестов для оценки скорости отклика (производительности) и стабильности системы. Эти тесты выполняются в среде предпроизводственной среде после развертывания кода после внесения существенных изменений. **Тестирование не фактического получения верного результата теста, но и скорости его получения.**

Это тесты, которые в идеале должен иметь каждый конвейер CI/CD, и, по сути, они должны быть автоматизированы. По мере того, как продукт растет и предлагает больше функций и обновлений, набор тестов также увеличиваются в геометрической прогрессии. Запустить все вышеперечисленные тесты вручную было бы невозможно, и автоматизация — единственный возможный способ выполнить их быстро и точно.

В составе CI/CD пайплайна место автоматизированных тестов находится между сборкой (на которой получаем работающий экземпляр ПО - контейнер или бинарный файл) и доставкой приложения на новый стенд, таким образом, при наличии проблем, обнаруженных автоматическими тестами, пайплайн обрывается и некачественный код не попадает ни на тестовый стенд, ни на производственный стенд.



Пример юнит-теста на языке Go:

```
package arraycounter //указываем тестируемый пакет

import "testing" //импортируем модуль для тестов - testing

func TestCountElementsInArray(t *testing.T){ /* Создаем функцию
                                                с именем Test+ТестируемаяФункция */

    got := CountElementsInArray([]int{1,2,2,3,4,6,6,6,1}, 6) /* Вызываем функцию
                                                                со статическими параметрами */

    want := 3 //Задаем эталонный результат функции

    if got != want { //сравниваем эталонный результат и реальный

        t.Errorf("got %q, wanted %q", got, want) /* если получившийся результат
                                                    не совпал с эталоном, вызываем ошибку*/
    }
}
```

Пример интеграционного теста на языке Go:

```
package arraycounter

import "testing"

func TestReplaceElementsInArray(t *testing.T){ /* Создаем функцию для тестирования
                                                нескольких функций внутри нее */

    marks, err := database.GetListOfMarksByStudentId(1) /* Вызываем первую
                                                            тестируемую функцию */

    if err != nil{ //Проверяем что функция выполнялась без ошибок
        t.Errorf("unable to get marks with error %s", err.Error())
    }

    got := CountElementsInArray(marks, 2) /* Проверяем что результат одной функции
                                                успешно обрабатывается другой функцией */

    want := 3

    if got != want {
        t.Errorf("got %q, wanted %q", got, want)
    }
}
```

Задания

1. Выбрать библиотеку, либо фреймворк для тестирования ПО на выбранном для разработки стеке
2. Разработать набор юнит- и интеграционных тестов для разработанной части ПО, таким образом, чтобы тестирование покрывало от 40% функционала, описанного в техническом задании. В отчете описать каждый отдельный тест (опционально — с примером кода), в частности, описать, какой модуль (модули) тестируется, какой ожидаемый результат, какие данные используются для получения искомого результата.
3. Добавить к разработанным конфигурациям сборки в TeamCity шаги, осуществляющие автоматическое тестирование разработанного кода (независимо от ветки, в которой код находится)
4. Продемонстрировать хотя бы одно неуспешное прохождение теста (с заранее плохим кодом, не тестом) и по одному успешному прохождению шагов тестирования для одной FEATURE/FIX ветки, DEV-ветки и PROD-ветки
5. Подготовить отчет по лабораторной работе (текстовый документ и скриншоты с описанием по вышеописанным пунктам).